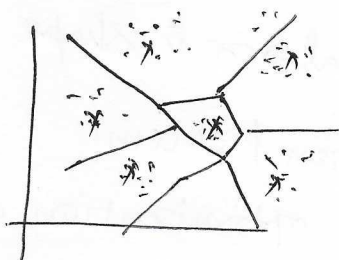


Clustering

- Grouping of data points
- Can be in higher dimension as well

K-means clustering

Unsupervised learning algorithm used to partition a dataset into k distinct groups/clusters such that points within a cluster are similar to each other, and points in different clusters are as dissimilar as possible.



- Get the number of clusters as input
- Randomly guess k cluster ~~DEFS~~ center
- Each datapoint finds out which center it's closest to.
- update the centroids
- Repeat until terminated.

Suppose cluster k contains points: $C_k = \{x_1, x_2, \dots\}$
 We want a single point that represents this cluster. The natural choice is the mean: $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$

Why mean? Because it minimizes squared distance.

$$\min_{\mu} \sum_{x_i \in C_k} \|x_i - \mu\|^2$$

So each cluster is represented by its centroid (mean)

Assigning points: for each point, assign a cluster index $C_i \in \{1, 2, \dots, k\}$

E.g. $C_5 = 3$ means point x_5 belongs to cluster #3

So, C_i tells which cluster point x_i belongs to.

Let μ_{C_i} is the centroid of the cluster assigned to point x_i

E.g. If $C_1 = 2$, $C_2 = 1$, then $\mu_{C_1} = \mu_2$, $\mu_{C_2} = \mu_1$

We want each point to be close to its cluster center. So for each point x_i , the error is: $\|x_i - \mu_{C_i}\|^2$

Total error: $J = \sum_{i=1}^N \|x_i - \mu_{C_i}\|^2$

Sometimes this is also written as,

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Algorithm Steps

1. Initialization: Choose k initial centroids
2. Assignment: Assign each point to the nearest centroid:
$$c_i = \arg \min_k \|x_i - \mu_k\|^2$$
3. Update: Recompute centroids:
$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$
4. Alternate between assignment & update until:
 - centroids stop changing
 - or - max^m iterations reached.

Limitations:

- Requires k in advance
- Sensitive to initialization
- Struggles with non-convex clusters
- Assumes clusters are roughly equal size & shape.

k-means actually solves the following optimization problem:

$$\min_{\{C_i\}, \{\mu_k\}} \sum_{i=1}^N \|x_i - \mu_{C_i}\|^2 \quad \leftarrow \text{Joint optimization over } C_i \text{ \& } \mu_k.$$

- Non-convex. No global optima guaranteed.

Alternating minimization

(Coordinate descent)

a) fix centroids, optimize assignments: $C_i = \arg \min_k \|x_i - \mu_k\|^2$

b) fix assignments, optimize centroids:

$$\text{solve } \mathcal{J}(\mu_k) = \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

$$\text{Now, } \|x_i - \mu_k\|^2 = (x_i - \mu_k)^T (x_i - \mu_k) = x_i^T x_i - 2x_i^T \mu_k + \mu_k^T \mu_k$$

$$\text{So, } \mathcal{J} = \sum_i x_i^T x_i - 2 \sum_i x_i^T \mu_k + \sum_i \mu_k^T \mu_k = \text{constant} - 2 \mu_k^T \sum_i x_i + n \mu_k^T \mu_k$$

$$\frac{\partial \mathcal{J}}{\partial \mu_k} = -2 \sum_i x_i + 2n \mu_k = 0$$

$$\Rightarrow \boxed{\mu_k = \frac{1}{n} \sum x_i}$$

- Squared distance penalizes points far away.
- The best balance point is the average location.

Hierarchical Clustering

A method of grouping similar data points into a tree-like structure (called dendrogram), where:

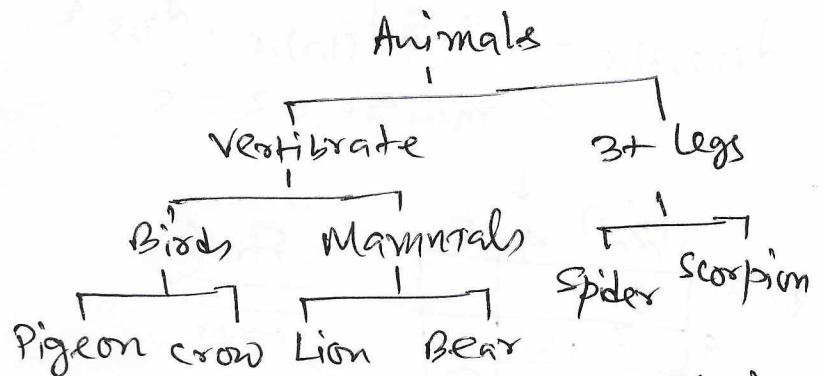
- each data point starts in its own cluster.
- pairs of clusters are merged based on similarity.
- this process continues until all points are in a single cluster (or until a stopping criteria is reached).

Two types of hierarchical clustering

- Agglomerative (bottom-up): starts with individual points & merge them step by step.
- Divisive (top-down): start with all data points in one cluster & divide them.

Agglomerative clustering

Begins with a distance matrix which contains the distances between every pair of objects.



start with each item in its own cluster, find the best pair to merge into a new cluster. Repeat until all clusters are fused together.

Consider all possible merges & choose the best.

↓
How to compute distances between cluster rather than objects?

Inter-cluster distance

- Single link: Distance of two closest members in each class.

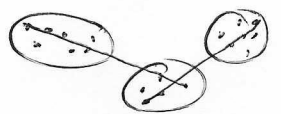
$$d_{\min}(C_i, C_j) = \min_{p \in C_i, q \in C_j} d(p, q)$$

- Complete link: Distance of two farthest members

$$d_{\max}(C_i, C_j) = \max_{p \in C_i, q \in C_j} d(p, q)$$

- Average link: average distance to all pairs

$$d_{\text{avg}}(C_i, C_j) = \frac{\sum_{p \in C_i, q \in C_j} d(p, q)}{|C_i| \cdot |C_j|}$$



Example (using single link)

	1	2	3	4	5
1	0				
2	2	0			
3	6	3	0		
4	10	9	7	0	
5	9	8	5	4	0

$$d_{(1,2),3} = \min \{d_{1,3}, d_{2,3}\} = \min \{6, 3\} = 3$$

$$\rightarrow d_{(1,2),4} = \min \{d_{1,4}, d_{2,4}\} = \min \{10, 9\} = 9$$

$$d_{(1,2),5} = \min \{d_{1,5}, d_{2,5}\} = \min \{9, 8\} = 8$$

1, 2 is min m.
Merge (1, 2)

$$d_{(1,2,3),4} = \min \{d_{(1,2),4}, d_{3,4}\} \\ = \min \{9, 7\} = 7$$

$$d_{(1,2,3),5} = \min \{d_{(1,2),5}, d_{3,5}\} \\ = \min \{8, 5\} = 5$$

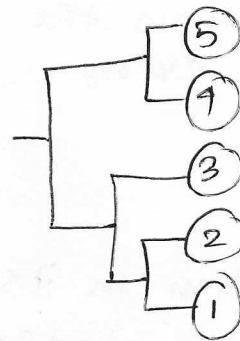
Merge (1, 2), 3

	(1, 2)	3	4	5
(1, 2)	0			
3	3	0		
4	9	7	0	
5	8	5	4	0

	(1, 2, 3)	4	5
(1, 2, 3)	0		
4	7	0	
5	5	4	0

Merge (4, 5)

Final
tree



Dendrogram

Heights
represent
distance
between
clusters.